College Campus Navigation System

C++ Source Code | campus.cpp

```cpp
#include <iostream>
#include <climits>
#include <cstring>
using namespace std;

#define MAX 10

class CampusGraph {
private:
    char location[MAX][30];
    int adj[MAX][MAX];
    int n;

    // Search location using Linear Search
    int searchLocation(char name[]) {
        for (int i = 0; i < n; i++) {
            if (strcmp(location[i], name) == 0)
                return i;
        }
        return -1;
    }

    // Recursive DFS function
    void dfsUtil(int v, int visited[]) {
        visited[v] = 1;
        cout << location[v] << " ";

        for (int i = 0; i < n; i++) {
            if (adj[v][i] > 0 && !visited[i]) {
                dfsUtil(i, visited);
            }
        }
    }

public:

    // Constructor
    CampusGraph() {
        n = 0;

        for (int i = 0; i < MAX; i++) {
            for (int j = 0; j < MAX; j++) {
                adj[i][j] = 0;
            }
        }
    }

    // Add new campus location
    void addLocation(char name[]) {
        if (n >= MAX) {
```

```
            cout << "Maximum locations reached!\n";
            return;
        }

        if (searchLocation(name) != -1) {
            cout << "Location already exists!\n";
            return;
        }

        strcpy(location[n], name);
        n++;

        cout << "Location added successfully!\n";
    }

    // Add road between two locations
    void addRoad(char a[], char b[], int distance) {
        int u = searchLocation(a);
        int v = searchLocation(b);

        if (u == -1 || v == -1 || u == v ||
            distance <= 0) {
            cout << "Invalid road details!\n";
            return;
        }

        adj[u][v] = distance;
        adj[v][u] = distance;

        cout << "Road added successfully!\n";
    }

    // Display all campus locations
    void displayLocations() {
        cout << "\nCampus Locations:\n";

        for (int i = 0; i < n; i++) {
            cout << i + 1 << ". "
                 << location[i] << endl;
        }
    }

    // Display connected locations
    void displayConnections() {
        cout << "\nCampus Connections:\n";

        for (int i = 0; i < n; i++) {
            cout << location[i] << " -> ";

            for (int j = 0; j < n; j++) {
```

```
                if (adj[i][j] > 0) {
                    cout << location[j]
                         << "(" << adj[i][j] << "m) ";
                }
            }
            cout << endl;
        }
    }

    // Search a particular location
    void searchPlace() {
        char name[30];

        cout << "Enter location name: ";
        cin >> name;

        int pos = searchLocation(name);

        if (pos == -1)
            cout << "Location not found!\n";
        else
            cout << "Location found: "
                 << location[pos] << endl;
    }

    // Breadth First Search
    void bfs(char start[]) {
        int s = searchLocation(start);

        if (s == -1) {
            cout << "Invalid starting location!\n";
            return;
        }

        int visited[MAX] = {0};
        int q[MAX];
        int front = 0, rear = 0;

        visited[s] = 1;
        q[rear++] = s;

        cout << "BFS Traversal: ";

        while (front < rear) {
            int v = q[front++];
            cout << location[v] << " ";

            for (int i = 0; i < n; i++) {
                if (adj[v][i] > 0 && !visited[i]) {
                    visited[i] = 1;
```

```
                    q[rear++] = i;
                }
            }
        }

        cout << endl;
    }

    // Depth First Search
    void dfs(char start[]) {
        int s = searchLocation(start);

        if (s == -1) {
            cout << "Invalid starting location!\n";
            return;
        }

        int visited[MAX] = {0};

        cout << "DFS Traversal: ";
        dfsUtil(s, visited);
        cout << endl;
    }

    // Dijkstra's Shortest Path Algorithm
    void shortestPath() {
        char source[30], destination[30];

        cout << "Enter starting location: ";
        cin >> source;

        cout << "Enter destination: ";
        cin >> destination;

        int s = searchLocation(source);
        int d = searchLocation(destination);

        if (s == -1 || d == -1) {
            cout << "Invalid location!\n";
            return;
        }

        int dist[MAX];
        int visited[MAX] = {0};
        int parent[MAX];

        for (int i = 0; i < n; i++) {
            dist[i] = INT_MAX;
            parent[i] = -1;
        }
```

```
201
202         dist[s] = 0;
203
204         for (int count = 0; count < n; count++) {
205             int u = -1;
206             int minimum = INT_MAX;
207
208             for (int i = 0; i < n; i++) {
209                 if (!visited[i] && dist[i] < minimum) {
210                     minimum = dist[i];
211                     u = i;
212                 }
213             }
214
215             if (u == -1)
216                 break;
217
218             visited[u] = 1;
219
220             for (int v = 0; v < n; v++) {
221                 if (adj[u][v] > 0 && !visited[v] &&
222                     dist[u] != INT_MAX &&
223                     dist[u] + adj[u][v] < dist[v]) {
224
225                     dist[v] = dist[u] + adj[u][v];
226                     parent[v] = u;
227                 }
228             }
229         }
230
231         if (dist[d] == INT_MAX) {
232             cout << "No path available!\n";
233             return;
234         }
235
236         int path[MAX];
237         int count = 0;
238         int current = d;
239
240         while (current != -1) {
241             path[count++] = current;
242             current = parent[current];
243         }
244
245         cout << "\nShortest Path: ";
246
247         for (int i = count - 1; i >= 0; i--) {
248             cout << location[path[i]];
249
250             if (i != 0)
```

```
                cout << " -> ";
        }

        cout << "\nTotal Distance: "
             << dist[d] << " meters\n";
    }
};

// Main Function
int main() {

    CampusGraph g;

    char name[30], a[30], b[30];
    int choice, distance;

    // Initial campus locations
    char places[8][30] = {
        "MainGate",
        "Library",
        "Canteen",
        "ComputerDept",
        "MechanicalDept",
        "AdminOffice",
        "Parking",
        "Auditorium"
    };

    for (int i = 0; i < 8; i++) {
        g.addLocation(places[i]);
    }

    // Initial campus roads
    g.addRoad((char*)"MainGate",
              (char*)"Library", 100);

    g.addRoad((char*)"MainGate",
              (char*)"Parking", 80);

    g.addRoad((char*)"Library",
              (char*)"Canteen", 60);

    g.addRoad((char*)"Library",
              (char*)"AdminOffice", 90);

    g.addRoad((char*)"Canteen",
              (char*)"ComputerDept", 70);

    g.addRoad((char*)"ComputerDept",
              (char*)"MechanicalDept", 110);
```

```
301
302     g.addRoad((char*)"AdminOffice",
303               (char*)"Auditorium", 120);
304
305     g.addRoad((char*)"Parking",
306               (char*)"Canteen", 150);
307
308     g.addRoad((char*)"MechanicalDept",
309               (char*)"Auditorium", 100);
310
311     do {
312         cout << "\n================================\n";
313         cout << " COLLEGE CAMPUS NAVIGATION SYSTEM\n";
314         cout << "================================\n";
315
316         cout << "1. Display Campus Locations\n";
317         cout << "2. Display Campus Connections\n";
318         cout << "3. Search Campus Location\n";
319         cout << "4. Find Shortest Path\n";
320         cout << "5. BFS Traversal\n";
321         cout << "6. DFS Traversal\n";
322         cout << "7. Add New Location\n";
323         cout << "8. Add New Road\n";
324         cout << "9. Exit\n";
325
326         cout << "\nEnter your choice: ";
327         cin >> choice;
328
329         switch (choice) {
330
331         case 1:
332             g.displayLocations();
333             break;
334
335         case 2:
336             g.displayConnections();
337             break;
338
339         case 3:
340             g.searchPlace();
341             break;
342
343         case 4:
344             g.shortestPath();
345             break;
346
347         case 5:
348             cout << "Enter starting location: ";
349             cin >> name;
350             g.bfs(name);
```

```
            break;

        case 6:
            cout << "Enter starting location: ";
            cin >> name;
            g.dfs(name);
            break;

        case 7:
            cout << "Enter new location: ";
            cin >> name;
            g.addLocation(name);
            break;

        case 8:
            cout << "Enter first location: ";
            cin >> a;

            cout << "Enter second location: ";
            cin >> b;

            cout << "Enter distance in meters: ";
            cin >> distance;

            g.addRoad(a, b, distance);
            break;

        case 9:
            cout << "Thank you for using the system!\n";
            break;

        default:
            cout << "Invalid choice!\n";
        }

    } while (choice != 9);

    return 0;
}
```